8. המתכנן הפעיל: RevealPlanner

`src/layerforge_l1/reveal_planner.py:237`. זה **מתכנן השכבות היחיד** שרץ ב־Option B, במצב `split_regions=True` ("per_cell"). לאותו קוד יש וריאנט השוואה, `split_regions=False` ("region_indivisible"), שבו כל אזור הוא תא אחד. הוא מתועד בסעיף 8.9 כי הוא חלק מהסקריפט ומהבדיקות, אבל ב־Option B הוא לא הורץ (`--variants per_cell`).

8.1 המודל: חשיפה מלפנים לאחור

IMPLEMENTED MATHEMATICS מסמנים את הלוחות לפי עומק $k = 0, 1, \ldots, S-1$ (0 = קדמי). לכל לוח יש צבע c_k וקבוצת תאים שהוא חושף, X_k, וכל תא ב־X_k הוא בצבע c_k. קבוצת התאים המכוסים לפני לוח k:

src/layerforge_l1/reveal_planner.py:348-351

```
348     def sheet_mask(self, covered_cells: np.ndarray, visible_cells: np.ndarray) -> np.ndarray:
349         covered = self.covered_mask(covered_cells | visible_cells)
350         visible = self.cells_mask(visible_cells)
351         return structural_support(covered, self.perimeter, self.kernel) | visible
```

DERIVED **האינטואיציה הפיזית.** פיקסל שכבר נחשף בלוח קדמי מוסתר מאחוריו, ולכן לוח k יכול להחזיק שם חומר בכל צבע (חומר **נסתר**) כדי לחזק את עצמו. פיקסל שעוד לא נחשף חייב להיות **חור** בלוח k, כי אחרת הוא יסתיר את הצבע שאמור להופיע מאחור. האינוריאנט $\mathrm{Struct}(U) \subseteq U$ (פרק 7) מבטיח בדיוק את זה: $M_k \subseteq \mathrm{px}(C_k \cup X_k)$.

DERIVED **משפט השחזור המדויק.** נניח שתא x נחשף בעומק d. לכל $k < d$, $x \notin C_k \cup X_k$, ולכן $x \notin M_k$. בעומק d, $x \in \mathrm{px}(X_d) \subseteq M_d$. לכן הלוח הקדמי ביותר שיש בו חומר ב־x הוא d, וצבעו $c_d = \mathrm{color}(x)$. **השחזור מדויק בהגדרה, בתנאי שכל התאים נחשפו.** ההרצה מאשרת זאת: ב־Option B יש 0 פיקסלים שונים.

8.2 unsupported_pixels: האם לוח מסוים בנוי היטב?

src/layerforge_l1/reveal_planner.py:273

src/layerforge_l1/reveal_planner.py:273-288

```
273     def unsupported_pixels(
274         self, covered_cells: np.ndarray, reveal: np.ndarray
275     ) -> np.ndarray:
276         """Visible pixels of ``reveal`` that the sheet cannot support."""
277 
278         visible = self.cells_mask(reveal)
279         covered = self.covered_mask(covered_cells) | visible
280         support = structural_support(covered, self.perimeter, self.kernel)
281         reach = _geodesic_reach(support, support | visible, self.nub_steps)
282         bad = visible & ~reach
283         if not bad.any():
284             # Nubs can rebuild a sub-width band the support had cut; check
285             # the finished sheet exactly as the validator will.
286             orphan = _orphan_cores(support | visible, self.perimeter, self.kernel)
287             bad = visible & cv2.dilate(orphan.astype(np.uint8), self.kernel).astype(bool)
288         return bad
```

IMPLEMENTED MATHEMATICS כאשר $V = \mathrm{px}(X)$ ו־$U = \mathrm{px}(C) \cup V$:

$$S=\mathrm{Struct}(U),\qquad \mathrm{Bad}=V\setminus R_5(S,\ S\cup V)$$

$$\text{if } \mathrm{Bad}=\emptyset:\quad \mathrm{Bad}=V\cap\Big(\big((S\cup V)\ominus K\big)\setminus\mathrm{Anch}_P\big((S\cup V)\ominus K\big)\Big)\oplus K$$

בדיקה ראשונה: כל פיקסל גלוי חייב להיות בתמיכה, או במרחק של עד 5 צעדים ממנה דרך פיקסלים גלויים. **בדיקה שנייה** (רק אם הראשונה עברה): הלוח הגמור $S \cup V$ נבדק בדיוק כמו ב־validator. אם יש ליבה יתומה, כל הפיקסלים הגלויים בטווח הגרעין ממנה נכשלים. הבדיקה הזאת נחוצה כי בליטות גלויות שמחוברות זו לזו יכולות לבנות מחדש "פס" צר שהתמיכה חתכה, וליצור ליבה יתומה.

8.3 maximal_reveal: החשיפה הגדולה ביותר (נקודת שבת)

src/layerforge_l1/reveal_planner.py:290

src/layerforge_l1/reveal_planner.py:290-304

```
290     def maximal_reveal(
291         self, covered_cells: np.ndarray, color: int
292     ) -> tuple[np.ndarray, int]:
293         """Greatest fixpoint of supportable uncovered cells of one color."""
294
295         candidate = (~covered_cells) & (self.cell_color == color)
296         iterations = 0
297         while candidate.any():
298             iterations += 1
299             bad = self.unsupported_pixels(covered_cells, candidate)
300             if not bad.any():
301                 break
302             bad_cells = np.unique(self.cell_map[bad])
303             candidate[bad_cells[bad_cells >= 0]] = False
304         return candidate, iterations
```

IMPLEMENTED MATHEMATICS

$$X^{(0)}=\{x:\ x\notin\mathcal C,\ \text{color}(x)=c\},\qquad X^{(i+1)}=X^{(i)}\setminus\mathrm{cells}\big(\mathrm{Bad}(\mathcal C,X^{(i)})\big)$$

מתחילים מ**כל** התאים בצבע c שעוד לא נחשפו, ומוחקים כל תא שיש בו לפחות פיקסל אחד לא נתמך, עד שאין יותר מחיקות. התוצאה יכולה להיות ריקה (אי אפשר לחשוף את הצבע הזה עכשיו).

DERIVED **למה זו "הגדולה ביותר"?** אם תא נכשל ב־$X^{(i)}$, אז בכל תת־קבוצה $Y \subseteq X^{(i)}$ התמיכה קטנה יותר (מונוטוניות), וגם קבוצת ההגעה קטנה יותר. לכן הוא ייכשל גם שם, וזה בטוח למחוק אותו. מכאן שהתוצאה מכילה כל קבוצת תאים חוקית בצבע c, כלומר זו **נקודת השבת הגדולה ביותר** (greatest fixpoint), והיא יחידה. CAVEAT הנימוק מניח מונוטוניות מלאה. הבדיקה השנייה (ליבות יתומות של הלוח הגמור) והתיקונים ההיוריסטיים אינם בהכרח מונוטוניים, ולכן זו הצדקת התכנון ולא הוכחה. כל לוח נבדק בסוף בכל מקרה על ידי ה־validator.

DERIVED **למה זה טוב לחשוף מקסימום?** אם חושפים יותר עכשיו, C של כל לוח מאחור גדל, ובזכות המונוטוניות התמיכה שלו רק גדלה. לכן חשיפה מקסימלית לא חוסמת אף אפשרות עתידית. זה מקטין את מרחב החיפוש: מכל מצב ולכל צבע יש בדיוק מעבר אחד.

8.4 closure: בדיקת היתכנות שלא תלויה בסדר

src/layerforge_l1/reveal_planner.py:353

```
src/layerforge_l1/reveal_planner.py:356-372
356         covered = self.start_covered.copy()
357         sequence: list[int] = []
358         progress = True
359         while progress and not covered.all():
360             progress = False
361             for color in self.colors:
362                 reveal, _ = self.maximal_reveal(covered, color)
363                 if reveal.any():
364                     covered |= reveal
365                     sequence.append(color)
366                     progress = True
367         return {
368             "feasible": bool(covered.all()),
369             "round_robin_sheet_count": len(sequence),
370             "obstruction_cells": np.flatnonzero(~covered).tolist(),
371             "covered_cells": covered,
372         }
```

Round-robin: עוברים על כל הצבעים שוב ושוב, ובכל פעם חושפים את המקסימום האפשרי לצבע, עד שאין התקדמות. אם בסוף C מכיל את כל התאים, הבעיה פתירה. אחרת, התאים שנשארו הם **קבוצת חסימה** (obstruction) מפורשת.

DERIVED **למה זה לא תלוי בסדר:** במודל מונוטוני, קבוצת התאים שאפשר לכסות היא הסגור (closure) של פעולת "חשוף מקסימום". כל סדר הוגן מגיע לאותה נקודת שבת: מה שנחשף בסדר אחד יישאר חשיף גם בסדר אחר, כי C רק גדל. לכן closure היא **תנאי הכרחי ומספיק** לקיום פתרון, בתוך המודל ובכפוף להסתייגות על מונוטוניות. ב־Option B: feasible = True, ב־12 לוחות round-robin, תוך 15.8 שניות (OBSERVED).

8.5 beam_search: מספר לוחות קטן

src/layerforge_l1/reveal_planner.py:374

src/layerforge_l1/reveal_planner.py:377-421

```
        start = self.start_covered.copy()
        beam: list[tuple[np.ndarray, list[int], list[np.ndarray]]] = [(start, [], [])]
        evaluations = 0
        for _ in range(self.config.max_sheets):
            expanded: dict[bytes, tuple[np.ndarray, list[int], list[np.ndarray]]] = {}
            for covered, colors, reveals in beam:
                for color in self.colors:
                    if colors and colors[-1] == color:
                        continue  # a repeat would have been revealed already
                    reveal, _ = self.maximal_reveal(covered, color)
                    evaluations += 1
                    if not reveal.any():
                        continue
                    nxt = covered | reveal
                    key = np.packbits(nxt).tobytes()
                    if key not in expanded:
                        expanded[key] = (nxt, colors + [color], reveals + [reveal])
            if not expanded:
                break
            complete = [state for state in expanded.values() if state[0].all()]
            if complete:
                covered, colors, reveals = complete[0]
                return {
                    "status": "COMPLETE",
                    "front_to_back_colors": colors,
                    "reveals": reveals,
                    "evaluations": evaluations,
                }
            beam = sorted(
                expanded.values(),
                key=self._beam_key,
            )[: self.config.beam_width]
        best = beam[0]
        return {
            "status": "INCOMPLETE",
            "front_to_back_colors": best[1],
            "reveals": best[2],
            "evaluations": evaluations,
            "uncovered_cells": np.flatnonzero(~best[0]).tolist(),
        }

    def _beam_key(self, state: tuple) -> tuple[int, int]:
        area = -int(self.cell_area[state[0]].sum())
        unfinished = len({int(c) for c in self.cell_color[~state[0]]})
        return (unfinished, area) if self.config.beam_score == "colors" else (area, unfinished)
```

איך זה עובד

- **מצב:** קבוצת המכוסים C, יחד עם רשימת הצבעים ורשימת החשיפות עד כה.
- **מעבר:** לכל צבע, מלבד הצבע של הלוח הקודם (שורה 384: לחזור על אותו צבע מיד אחריו מיותר, כי החשיפה הקודמת כבר הייתה מקסימלית), מבצעים `maximal_reveal`. אם יצא ריק, מדלגים.
- **ביטול כפילויות (שורה 391):** `src/layerforge_l1/reveal_planner.py:391`. שני מסלולים שמגיעים לאותה קבוצה C שקולים לגבי העתיד, ולכן נשמר רק הראשון.
- **עצירה:** (א) נמצא מצב שכל התאים בו מכוסים, ומוחזר הראשון ברשימה, COMPLETE. (ב) אין הרחבות, כלומר כל הצבעים נתקעו. (ג) הגיעו ל־`max_sheets=24`. ב־(ב) וב־(ג) מוחזר INCOMPLETE עם המצב הטוב ביותר.
- **דירוג:** רק `beam_width` המצבים הטובים ביותר עוברים לשלב הבא.

IMPLEMENTED MATHEMATICS מפתח הדירוג (`src/layerforge_l1/reveal_planner.py:418`). ב־`"colors"` (Option B) ממיינים לקסיקוגרפית, מהקטן לגדול:

$$\kappa(\mathcal C)=\Big(\ \big|\{\text{color}(x):x\notin\mathcal C\}\big|\ ,\ -\!\!\sum_{x\in\mathcal C}\text{area}(x)\ \Big)$$

קודם מעדיפים מצבים שבהם פחות צבעים "לא גמורים", כלומר צבעים שעדיין יש מהם תאים חשופים. בשוויון מעדיפים את המצב שכיסה יותר שטח. ב־`"area"` הסדר הפוך.

DERIVED **אילו הבטחות יש ואילו אין.** החיפוש מתקדם רמה אחר רמה (BFS עם גיזום), ולכן הפתרון הראשון שנמצא הוא הקצר ביותר **מבין המצבים ששרדו את הגיזום**. UNVERIFIED אין הבטחת אופטימליות גלובלית: מצב שנגזם יכול היה להוביל לפתרון קצר יותר. חסם תחתון טריוויאלי **שכן** מוכח: כל צבע צריך לוח אחד לפחות, ולכן $S \geq 9$ ב־Option B. הטענה ב־STATUS.md ש־10 הוא החסם התחתון נשענת על טיעון ידני (השחור חייב להופיע גם לפני וגם אחרי אזור אחר). הקוד לא מוכיח אותה.

IMPLEMENTED MATHEMATICS **מספר ההערכות.** ברמה הראשונה יש $|colors| = 9$ קריאות. בכל רמה אחרת יש לכל היותר $8 \cdot 8 = 64$. לכן לפתרון של 10 לוחות: $B \cdot (|colors| - 1) \leq 9 + 9 \cdot 64 = 585$. בפועל נרשמו 553 (OBSERVED).

8.6 eliminate_sheets: מחיקת לוחות בדיעבד

`src/layerforge_l1/reveal_planner.py:306`. ננסה להעביר את כל תאי לוח מסוים ללוח מאוחר יותר באותו צבע. לוחות קטנים נבדקים ראשונים. כל הלוחות שביניהם מאבדים את התאים האלה מקבוצת המכוסים שלהם, ולכן כל אחד מהם נבדק מחדש ב־`unsupported_pixels`. המהלך נשמר רק אם כולם תקינים. אחרי כל הצלחה מתחילים מחדש. **ב־Option B לא נמחק אף לוח** (`eliminated_sheets: []`).

```
src/layerforge_l1/reveal_planner.py:315-346
315         colors, reveals = list(colors), [r.copy() for r in reveals]
316         log: list[dict[str, Any]] = []
317         changed = True
318         while changed:
319             changed = False
320             order = sorted(range(len(reveals)), key=lambda j: int(self.cell_area[reveals[j]].sum()))
321             for source in order:
322                 for target in range(source + 1, len(reveals)):
323                     if colors[target] != colors[source]:
324                         continue
325                     trial = [r.copy() for r in reveals]
326                     trial[target] |= trial[source]
327                     trial[source][:] = False
328                     covered = self.start_covered.copy()
329                     valid = True
330                     for depth in range(len(trial)):
331                         if depth >= source and trial[depth].any():
332                             if self.unsupported_pixels(covered, trial[depth]).any():
333                                 valid = False
334                                 break
335                         covered |= trial[depth]
336                     if valid:
337                         log.append({"removed_front_to_back_depth": source, "merged_into": target,
338                                     "color": colors[source], "cells_moved": int(reveals[source].sum())})
339                         del trial[source]
340                         del colors[source]
341                         reveals = trial
342                         changed = True
343                         break
344                 if changed:
345                     break
346         return colors, reveals, log
```

8.7 materialize ו־seam_report

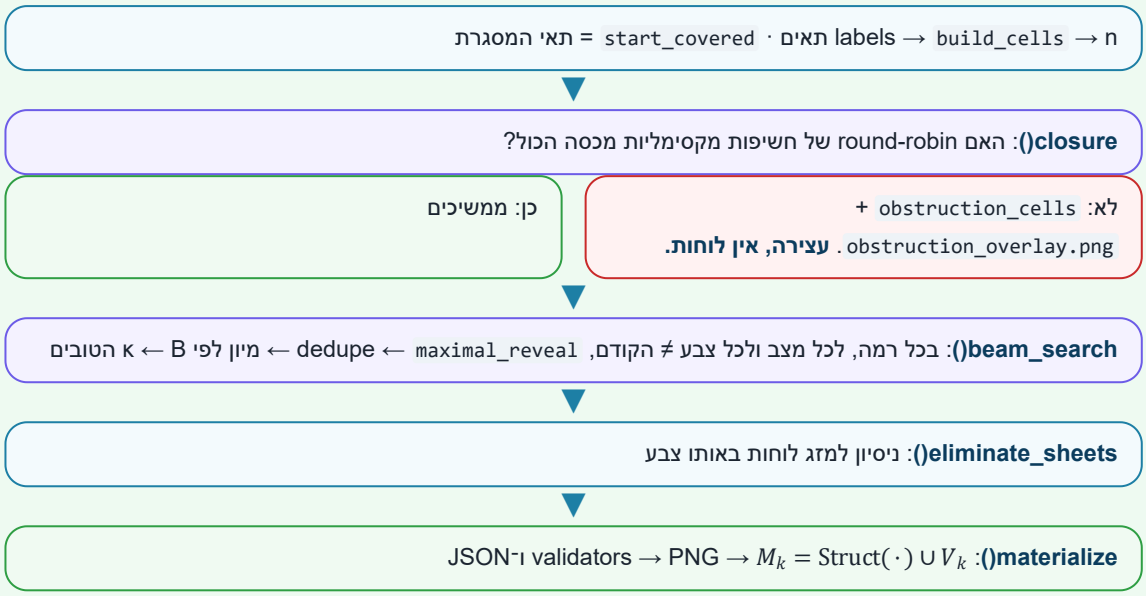
```
src/layerforge_l1/reveal_planner.py:426-438
426         covered = self.start_covered.copy()
427         sheets = []
428         for reveal in reveals:
429             physical = self.sheet_mask(covered, reveal)
430             sheets.append(
431                 {
432                     "physical_mask": physical,
433                     "visible_mask": self.cells_mask(reveal),
434                     "visible_cells": reveal.copy(),
435                 }
436             )
437             covered |= reveal
438         return sheets
```

`materialize` עובר על החשיפות מלפנים לאחור ובונה לכל לוח את M_k ואת V_k. `src/layerforge_l1/reveal_planner.py:440` סופר זוגות של פיקסלי יעד סמוכים (אופקית ואנכית) שנמצאים בתאים שונים **של אותו אזור** ובעומקים שונים:

$$L_{\text{seam}}=s\cdot\big|\{(a,b)\ \text{4-adjacent:}\ \mathrm{cell}(a)\neq\mathrm{cell}(b),\ \mathrm{reg}(a)=\mathrm{reg}(b),\ \mathrm{depth}(a)\neq\mathrm{depth}(b)\}\big|$$

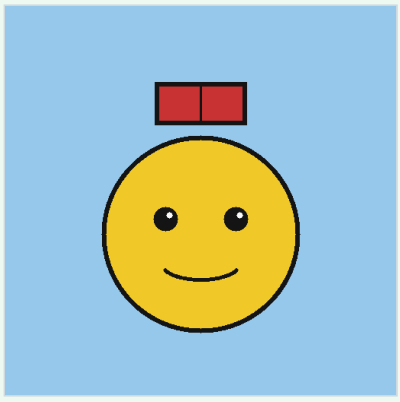
ב־Option B: 171 זוגות, כלומר $171 \cdot 0.09765625 = 16.7$ מ"מ. רק אזור אחד (מזהה 0, השחור) פוצל בין שני עומקים.

8.8 תרשים 3: זרימת יצירת השכבות

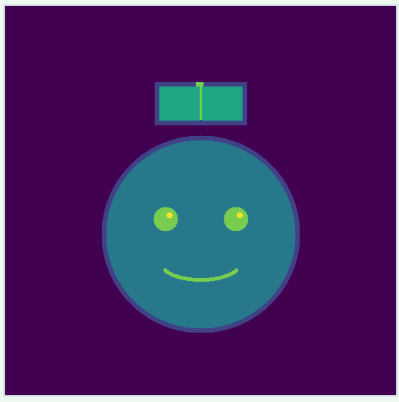


8.9 דוגמת ריצה: הפיקסטורה הסינתטית מהבדיקות

הפיקסטורה `tests/test_reveal_planner.py:20` היא איור מקורי (לא מוגן זכויות): פרצוף צהוב עם מתאר שחור, ומעליו פפיון עם מתאר שחור, שתי אונות אדומות ומחיצה שחורה ברוחב 3 פיקסלים בלבד (1.17 מ"מ ברשת של 512 פיקסלים). הרצתי את המתכנן האמיתי עם `RevealConfig(scale=2)`, כמו בבדיקה (OBSERVED), ב־20.7 שניות:

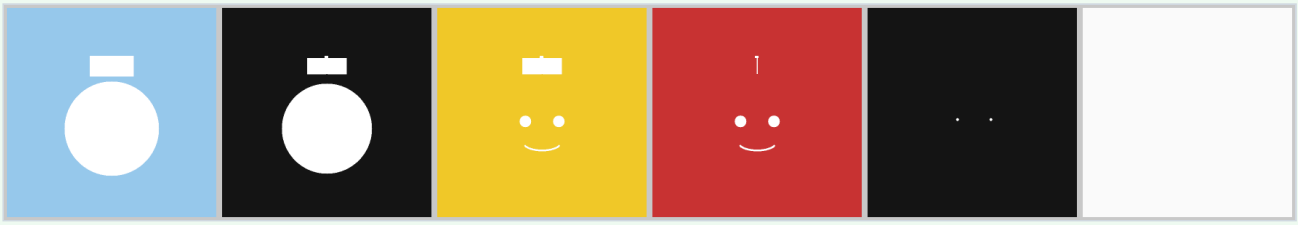


יעד 512 × 512 (צבעים: 0 = שמיים, 1 = שחור, 2 = צהוב, 3 = אדום, 4 = לבן)



עומק החשיפה לכל פיקסל (כהה = קדמי)

נתון	ערך
רשת	outer = 1152 פיקסלים, $s = 225/1152 = 0.1953$ מ"מ, גרעין 7×7, nub = 3 צעדים
תאים	2949 (במצב region_indivisible: 11 אזורים)
חשיפה מקסימלית בצעד הראשון	שמיים: 2156/2156 תאים (איטרציה אחת). שחור, צהוב, אדום ולבן: 0. בלוח הקדמי השמיים מסביבם עדיין לא חשופים, ולכן חייבים להיות חור. בלי שום חומר נסתר הם היו "איים" שלא מחוברים למסגרת.
closure	feasible, 6 לוחות round-robin
beam	COMPLETE אחרי 49 הערכות. מלפנים לאחור: 0, 1, 2, 3, 1, 4, עם 2156, 153, 512, 74, 51 ו־3 תאים
לוחות	6 מתוך 6 עוברים neighbor-4 ו־width. חומר שנשאר: 0.819, 0.839, 0.979, 0.993, 1.0, 1.0
שחזור	0 פיקסלים שונים
תפרים	אזור אחד מפוצל, 15 זוגות = 5.86 מ"מ
וריאנט region_indivisible	**לא פתיר**: closure מחזירה 3 אזורי חסימה



ששת הלוחות הפיזיים, משמאל לימין: עומק 0 (קדמי) עד עומק 5 (אחורי). השחור מופיע פעמיים: בלוח 1 (המתאר החיצוני) ובלוח 4 (המחיצה, שמאחורי האדום)

DERIVED **למה השחור צריך שני לוחות:** אם המחיצה השחורה (1.17 מ"מ) באותו לוח עם המתאר, לפני האדום, היא פס צר מ־1.27 מ"מ ונפסלת. אם היא מאחורי האדום כחלק מאזור שלם, האונות האדומות נשארות "איים" בלוח האדום. פיצול לתאים מאפשר למתאר להיחשף מוקדם ולמחיצה מאוחר יותר, ואז לוח האדום מחזיק את החומר הנסתר שמאחורי המתאר. זו הסיבה לתוצאה `count(BLACK) == 2` שבבדיקה.

8.10 תשובות לשאלות היסוד מתוך הקוד

שאלה	תשובה
כמה לוחות?	מספר הרמות עד שה־beam מוצא מצב מכוסה במלואו, ואחר כך מחיקות ב־eliminate. אין קלט "מספר לוחות".
איזה צבע לכל לוח?	הצבע שבחר המעבר ב־beam. כל לוח הוא צבע אחד, ומותר שצבע יחזור (לא ברצף).
סדר מלפנים לאחור?	סדר הרמות ב־beam: לוח 0 נבחר ראשון. בפלט המספור הפוך (`back_index = S-1-depth`).
מה נשאר חומר?	$M_k = \mathrm{Struct}(\mathrm{px}(C_k \cup X_k)) \cup V_k$.
היכן החורים?	כל מה שמחוץ ל־M_k: כל התאים שעוד לא נחשפו, וגם חומר נסתר דק שהתמיכה השמיטה.
איך צבע אחורי נראה?	לכל k קודם, התא שלו מחוץ ל־$C_k \cup X_k$, ולכן חור בכל הלוחות שלפניו (8.1).
חומר נסתר?	$M_k \cap \mathrm{px}(C_k)$: גלוי בלוח קדמי ולכן מוסתר, ומשמש כ"שלד" שמחבר את V_k למסגרת.
גאומטריה דקה או מנותקת?	גלויה: נדחית (התא חוזר לרשימה ויחכה לשלב שבו יש לו תמיכה). נסתרת: מושמטת בשקט, וזה בטוח.
קבלה ודחייה?	תא מתקבל רק אם כל הפיקסלים שלו עוברים את `unsupported_pixels`. לוח שלם עובר בסוף את ה־validator.
האם משנה גאומטריה?	לא. היעד לא משתנה. מתווסף רק חומר נסתר (תחת לוחות קדמיים).
מתי עוצר?	COMPLETE, תקיעה, או 24 רמות.
אם אין פתרון?	closure = False: אין לוחות, יש `obstruction_overlay.png` ופירוט לפי צבע. beam INCOMPLETE: הסקריפט עדיין מייצר לוחות חלקיים, והשחזור ייכשל (mismatch > 0).

8.11 סיבוכיות ומקורות העלות

DERIVED

תהי T_u העלות של קריאה ל־`unsupported_pixels` (כמה פעולות מורפולוגיות על 2304²), I מספר האיטרציות בנקודת השבת, B רוחב ה־beam, Q מספר הצבעים ו־S מספר הלוחות.

$$T_{\text{beam}}\approx O\big(S\cdot B\cdot Q\cdot \bar I\cdot T_u\big),\qquad T_{\text{closure}}\approx O\big(R\cdot Q\cdot\bar I\cdot T_u\big)$$

R הוא מספר הסבבים. ב־Option B היו 553 הערכות, וכל המתכנן (כולל closure, validators וכתיבת PNG) רץ 304 שניות. כלומר, פחות מ־0.55 שניות להערכה בממוצע. הזיכרון: כל מצב שומר רשימת וקטורי bool באורך n (11,498), כך שהוא זניח לעומת המסכות. כל מסכה במלוא הרשת תופסת כ־5.3 מגה־בייט, וכמה כאלה קיימות בו־זמנית בכל קריאה. `eliminate_sheets` עולה במקרה הגרוע $O(S^3)$ בדיקות.

8.12 תנאי כשל

- **מוכח בבדיקה:** במצב region_indivisible הפיקסטורה לא פתירה. במצב per_cell היא כן.
- **מוכח בהיסטוריה:** שוליים של 2 פיקסלים בגרעין (`planning_margin_pixels`) הגדילו את מספר הלוחות (22 לוחות בצב), ובוטלו.
- **תיאורטי:** פרט גלוי צר מ־1.27 מ"מ שאינו "בליטה" קצרה (למשל קו דק וארוך) לא יכול להיתמך בשום עומק, כי התמיכה שלו תמיד דקה מדי. closure תחזיר אותו כחסימה. זו הסיבה לתיקון הידני של הלחי (Option A).

- **תיאורטי:** מכיוון שה־beam לא אופטימלי, ייתכן פתרון קצר יותר שלא נמצא.
- **תפעולי:** זיכרון וזמן גדלים עם רוחב ה־beam. הרצה עם beam 16 על הצב נעצרה בגלל לחץ זיכרון במחשב.